

Pack 24 (Web + API) — Admin RBAC on Trust/Grant Mutations + Server-Side Receipt Verification (real code)

RemoteDesk · Pack 24 (Web + API) — Admin RBAC on Trust/Grant Mutations + Server-Side Receipt Verification (real code)

Codex / Claude Code handoff. Direct continuation of Pack 23. Pack 23 left two flags: trust/grant mutations were auth-only (any org member could trust a device or mint a grant), and receipt POST trusted the desktop blindly. This slice adds (1) an admin RBAC guard so only owners/admins can change trust or create/revoke grants, and (2) server-side receipt verification, the API recomputes the consent receipt's hash chain before storing, rejects tampered/over-claiming receipts, and exposes a tamper-check endpoint. Build order: shared rbac/verify helpers → RBAC middleware → receipt verifier service → route guards → web gating → tests. Scope: lock down who can mutate trust/grants, and make stored receipts provably intact + content-free. Reuses Pack 6 RBAC (`can` /permissions) where present. **Server clamp still applies; host still owns live permissions. No secrets/payloads accepted into receipts. Fail-closed: unknown role => denied; unverifiable receipt => rejected.**

Tag legend: **[NEW]** = create file. **[MODIFY]** = patch existing file by hand (merge, don't overwrite).

Files created / modified / not touched

CREATED [NEW]	
packages/shared/src/permissions/adminRbac.ts	
packages/shared/src/permissions/receiptVerification.ts	
apps/api/src/middleware/requireDeviceAdmin.middleware.ts	
apps/api/src/services/admin/receiptVerifier.service.ts	
apps/api/src/lib/nodeSha256.ts	
apps/web/app/dashboard/devices/useDeviceAdminPermissions.ts	
tests/admin/adminRbac.test.ts	
tests/admin/receiptVerification.test.ts	
tests/admin/receiptVerifier.service.test.ts	
MODIFIED [MODIFY]	
packages/shared/src/permissions/index.ts	(export adminRbac + receiptVerification)
apps/api/src/routes/deviceAdmin.routes.ts	(guard mutations + verify on receipt store)
apps/web/app/dashboard/devices/[deviceId]/page.tsx	(hide trust/grant controls for non-admins)
apps/web/app/dashboard/receipts/page.tsx	(per-row verified badge + re-verify)
NOT TOUCHED (intentionally)	
apps/desktop/**	(desktop already builds receipts the same way)
apps/api/prisma/**	(no schema change; verification is computed)
apps/web/app/dashboard/devices/page.tsx	(list view unchanged)

Part A — Shared: admin RBAC + receipt verification helpers

FILE: packages/shared/src/permissions/adminRbac.ts

```
/**
 * Device-admin authorization. Reuses the org role model from Pack 6 (owner/admin/
 * member). Trust changes + grant create/revoke are admin-or-owner only. Pure +
 * fail-closed: an unknown/missing role denies.
 */
export type OrgRoleLike = 'owner' | 'admin' | 'member' | string | null | undefined;

export type DeviceAdminAction =
  | 'device.trust.set'
  | 'device.grant.create'
  | 'device.grant.revoke';

/** Roles permitted to mutate device trust/grants. */
const ADMIN_ROLES = new Set(['owner', 'admin']);

export function canMutateDeviceAdmin(role: OrgRoleLike): boolean {
  return typeof role === 'string' && ADMIN_ROLES.has(role);
}

/** Per-action check (all device-admin actions currently require admin/owner). */
export function canPerformDeviceAdminAction(role: OrgRoleLike, _action: DeviceAdminAction): boolean {
  return canMutateDeviceAdmin(role);
}

/** Read access: any member of the org may VIEW devices/receipts. */
export function canViewDeviceAdmin(role: OrgRoleLike): boolean {
  return typeof role === 'string' && (ADMIN_ROLES.has(role) || role === 'member');
}
```

FILE: packages/shared/src/permissions/receiptVerification.ts

```
import { verifyReceipt, type ConsentReceipt } from './consentReceipt';
import type { SessionSummary } from './sessionSummary';

/**
 * Server-side receipt validation: structural (content-free) checks + hash-chain
 * verification. The chain check reuses the SAME verifyReceipt from Pack 22, so
 * the desktop and server agree on what "intact" means.
 */

/** Keys that must NOT appear anywhere in a receipt (content-free invariant). */
const FORBIDDEN_KEYS = [
  /payload/i, /keycode/i, /coords?/i, /clipboard/i, /filename/i, /filecontent/i,
  /token/i, /password/i, /secret/i, /authorization/i, /cookie/i, /\xbip\b/i,
];

/** Deep scan: returns the first forbidden key found, or null. */
export function findForbiddenKey(value: unknown, path = ''): string | null {
  if (!value || typeof value !== 'object') return null;
  for (const [k, v] of Object.entries(value as Record<string, unknown>)) {
    if (FORBIDDEN_KEYS.some((re) => re.test(k))) return path ? `${path}.${k}` : k;
    const nested = findForbiddenKey(v, path ? `${path}.${k}` : k);
    if (nested) return nested;
  }
}
```

```

    }
    return null;
}

export type ReceiptRejectReason =
  | 'bad_version'
  | 'missing_fields'
  | 'forbidden_content'
  | 'summary_mismatch'
  | 'chain_invalid';

export interface ReceiptValidation {
  valid: boolean;
  reason: ReceiptRejectReason | null;
  firstBrokenSeq: number | null;
}

/** Structural checks that need no hashing. */
export function structuralCheck(receipt: ConsentReceipt): ReceiptRejectReason | null {
  if (receipt.version !== 1) return 'bad_version';
  if (!receipt.receiptId || !receipt.rootHash || !receipt.summary || !Array.isArray(receipt.lineItems)) {
    return 'missing_fields';
  }
  if (findForbiddenKey(receipt)) return 'forbidden_content';
  if (!summaryConsistent(receipt.summary, receipt)) return 'summary_mismatch';
  return null;
}

/** Summary counts must not exceed what the line items can support (anti-overclaim). */
function summaryConsistent(summary: SessionSummary, receipt: ConsentReceipt): boolean {
  const lineCount = receipt.lineItems.length;
  const c = summary.counts;
  const totalClaimed = c.inputBlocked + c.inputRateLimited + c.emergencyStops + c.controlGrants + c.controlRevokes;
  // Claimed events can't exceed the number of timeline lines.
  return totalClaimed <= lineCount;
}

/**
 * Full validation: structural first (cheap), then hash-chain via injected sha256.
 * Fail-closed: any structural failure short-circuits before hashing.
 */
export async function validateReceipt(
  receipt: ConsentReceipt,
  sha256Hex: (input: string) => Promise<string>,
): Promise<ReceiptValidation> {
  const structural = structuralCheck(receipt);
  if (structural) return { valid: false, reason: structural, firstBrokenSeq: null };

  const chain = await verifyReceipt(receipt, sha256Hex);
  if (!chain.valid) return { valid: false, reason: 'chain_invalid', firstBrokenSeq: chain.firstBrokenSeq };

  return { valid: true, reason: null, firstBrokenSeq: null };
}

```

```
// ...existing exports through Pack 23...
export * from './adminRbac';           // ADD
export * from './receiptVerification'; // ADD
```

Part B — API: RBAC middleware

FILE: apps/api/src/middleware/requireDeviceAdmin.middleware.ts

```
import type { Request, Response, NextFunction } from 'express';
import { AppError, ErrorCodes, canMutateDeviceAdmin } from '@remotedesk/shared';
import type { OrgRequest } from './orgContext.middleware';

/**
 * Guards device-admin MUTATIONS (trust set, grant create/revoke). Relies on
 * orgContext having resolved the caller's org role (Pack 6). Fail-closed: a
 * missing/unknown role is denied.
 */
export function requireDeviceAdmin(req: Request, _res: Response, next: NextFunction): void {
  const role = (req as OrgRequest & { orgRole?: string }).orgRole;
  if (!canMutateDeviceAdmin(role)) {
    return next(new AppError(ErrorCodes.PERMISSION_DENIED));
  }
  next();
}
```

Part C — API: node sha256 + receipt verifier service

FILE: apps/api/src/lib/nodeSha256.ts

```
import { createHash } from 'node:crypto';

/**
 * Server SHA-256 (hex) matching the desktop's Web Crypto output. Used to verify
 * consent-receipt hash chains identically to how the desktop built them.
 */
export async function nodeSha256Hex(input: string): Promise<string> {
  return createHash('sha256').update(input).digest('hex');
}
```

FILE: apps/api/src/services/admin/receiptVerifier.service.ts

```
import {
  AppError,
  ErrorCodes,
  validateReceipt,
  type ConsentReceipt,
  type ReceiptValidation,
} from '@remotedesk/shared';
import { nodeSha256Hex } from '../../../lib/nodeSha256';
import { consentReceiptRepository } from '../../../repositories/consentReceiptRepository';

/**
 * Verifies consent receipts server-side before storing, and re-verifies stored
```

```

* ones on demand. A receipt that fails structural OR chain checks is rejected and
* never stored. Verification reuses the shared validator with the node hash.
*/
export const receiptVerifierService = {
  /** Validate + store. Throws if invalid (caller returns 400). */
  async storeVerified(orgId: string, receipt: ConsentReceipt): Promise<ReceiptValidation> {
    const result = await validateReceipt(receipt, nodeSha256Hex);
    if (!result.valid) throw new AppError(ErrorCodes.RECEIPT_INVALID);
    await consentReceiptRepository.store(orgId, receipt.summary.deviceId, receipt);
    return result;
  },

  /** Re-verify a previously stored receipt (tamper check at rest). */
  async verifyStored(orgId: string, receiptId: string): Promise<ReceiptValidation> {
    const receipt = await consentReceiptRepository.getJson(orgId, receiptId);
    if (!receipt) throw new AppError(ErrorCodes.RECEIPT_NOT_FOUND);
    return validateReceipt(receipt, nodeSha256Hex);
  },
};

```

Add error codes (REVIEW_REQUIRED — patch `packages/shared/src/errors/errorCodes.ts`):

```

RECEIPT_INVALID: 'RECEIPT_INVALID',
RECEIPT_NOT_FOUND: 'RECEIPT_NOT_FOUND',

```

Part D — API: route guards

FILE: `apps/api/src/routes/deviceAdmin.routes.ts` [MODIFY]

```

// ...existing Pack 23 imports...
import { requireDeviceAdmin } from '../middleware/requireDeviceAdmin.middleware';
import { receiptVerifierService } from '../services/admin/receiptVerifier.service';

// ---- MODIFY: add requireDeviceAdmin to the three MUTATION routes ----
// .patch('/:deviceId/trust', requireAuth, orgContext, requireDeviceAdmin, h(async (req, res) => { ... })))
// .post('/:deviceId/grants', requireAuth, orgContext, requireDeviceAdmin, h(async (req, res) => { ... })))
// .delete('/:deviceId/grants/:grantId', requireAuth, orgContext, requireDeviceAdmin, h(async (req, res) => { ... })))
//
// GET routes (list/detail/grants/receipts) stay requireAuth + orgContext only
// (any org member may view).

// ---- MODIFY: receipt store now VERIFIES before persisting ----
// .post('/receipts', requireAuth, orgContext, h(async (req, res) => {
//   const receipt = req.body; // content-free ConsentReceipt from the desktop
//   const result = await receiptVerifierService.storeVerified((req as OrgRequest).orgId, receipt);
//   res.status(201).json({ stored: true, receiptId: receipt.receiptId, valid: result.valid });
// })))

// ---- ADD: a tamper-check endpoint for a stored receipt ----
// .get('/receipts/:receiptId/verify', requireAuth, orgContext, h(async (req, res) => {
//   res.json(await receiptVerifierService.verifyStored((req as OrgRequest).orgId, req.params.receiptId));
// })))

```

Part E — Web: permission gating

FILE: apps/web/app/dashboard/devices/useDeviceAdminPermissions.ts

```
import { canMutateDeviceAdmin, type OrgRoleLike } from '@remotedesk/shared';

/**
 * Renderer-side gate mirroring the server's requireDeviceAdmin, so the UI hides
 * controls the API would reject. Server remains authoritative; this is UX only.
 */
export function useDeviceAdminPermissions(role: OrgRoleLike) {
  return {
    canMutate: canMutateDeviceAdmin(role),
  };
}
```

FILE: apps/web/app/dashboard/devices/[deviceId]/page.tsx [MODIFY]

```
// ...existing Pack 23 imports...
import { useDeviceAdminPermissions } from '../useDeviceAdminPermissions';

// ---- ADD: resolve the caller's org role (from your existing auth/session context) ----
// const { role } = useOrgSession();          // your existing hook
// const { canMutate } = useDeviceAdminPermissions(role);

// ---- MODIFY: gate the trust buttons + grant form on canMutate ----
// {canMutate ? (
//   <section className={styles.panel}>
//     <h2>Trust</h2>
//     <div className={styles.row}>
//       <button className={styles.btn} onClick={() => setTrust('trusted')}>Trust</button>
//       <button className={styles.btn} onClick={() => setTrust('untrusted')}>Untrust</button>
//       <button className={styles.btnDanger} onClick={() => setTrust('blocked')}>Block</button>
//     </div>
//   </section>
// ) : (
//   <section className={styles.panel}><p className={styles.muted}>Only admins can change device trust.</p></section>
// )}
//
// Similarly: render the "Create grant" form + per-row Revoke buttons only when canMutate.
// The grants TABLE (read) stays visible to everyone.
```

FILE: apps/web/app/dashboard/receipts/page.tsx [MODIFY]

```
// ...existing Pack 23 imports...
import { api } from '../../../src/api/apiClient';
import type { ReceiptValidation } from '@remotedesk/shared';

// ---- ADD: per-receipt re-verify (tamper check at rest) ----
// const [verified, setVerified] = useState<Record<string, ReceiptValidation>>({});
// const verify = async (receiptId: string) => {
//   const v = await api.get<ReceiptValidation>(`/devices/receipts/${receiptId}/verify`);
//   setVerified((prev) => ({ ...prev, [receiptId]: v }));
// };

// ---- ADD a column + button to the table ----
// <td>
//   <button className={styles.btn} onClick={() => verify(r.receiptId)}>
```

```
//      {verified[r.receiptId] ? (verified[r.receiptId].valid ? '✓ intact' : '☒ tampered') : 'Verify'}  
//    </button>  
//  </td>
```

Part F — Tests

FILE: tests/admin/adminRbac.test.ts

```
import { describe, it, expect } from 'vitest';  
import { canMutateDeviceAdmin, canViewDeviceAdmin, canPerformDeviceAdminAction } from '@remotedesk/shared';  
  
describe('adminRbac', () => {  
  it('owner + admin may mutate; member may not', () => {  
    expect(canMutateDeviceAdmin('owner')).toBe(true);  
    expect(canMutateDeviceAdmin('admin')).toBe(true);  
    expect(canMutateDeviceAdmin('member')).toBe(false);  
  });  
  
  it('fails closed on unknown/missing role', () => {  
    expect(canMutateDeviceAdmin(null)).toBe(false);  
    expect(canMutateDeviceAdmin(undefined)).toBe(false);  
    expect(canMutateDeviceAdmin('guest')).toBe(false);  
  });  
  
  it('any member may view', () => {  
    expect(canViewDeviceAdmin('member')).toBe(true);  
    expect(canViewDeviceAdmin('admin')).toBe(true);  
    expect(canViewDeviceAdmin(null)).toBe(false);  
  });  
  
  it('per-action check matches mutate rule', () => {  
    expect(canPerformDeviceAdminAction('admin', 'device.trust.set')).toBe(true);  
    expect(canPerformDeviceAdminAction('member', 'device.grant.create')).toBe(false);  
  });  
});
```

FILE: tests/admin/receiptVerification.test.ts

```
import { describe, it, expect } from 'vitest';  
import { createHash } from 'node:crypto';  
import {  
  validateReceipt,  
  structuralCheck,  
  findForbiddenKey,  
  buildChain,  
  makeAuditEvent,  
  type ConsentReceipt,  
  type SessionSummary,  
} from '@remotedesk/shared';  
  
const sha256Hex = async (i: string) => createHash('sha256').update(i).digest('hex');  
const events = [makeAuditEvent('control.granted'), makeAuditEvent('input.blocked'), makeAuditEvent('control.revoked')];  
  
const summary: SessionSummary = {  
  deviceId: 'dev-1', startedAt: '2026-06-15T10:00:00.000Z', endedAt: '2026-06-15T10:05:00.000Z',
```

```

    durationSeconds: 300, capabilitiesUsed: { remoteInput: true, clipboardSync: false, fileTransfer: false },
    unattended: false, counts: { inputBlocked: 1, inputRateLimited: 0, emergencyStops: 0, controlGrants: 1, controlRevokes:
1 },
    endReason: 'user_disconnect',
  };

async function receipt(): Promise<ConsentReceipt> {
  const { lineItems, rootHash } = await buildChain(events, sha256Hex);
  return { receiptId: 'r1', summary, lineItems, rootHash, issuedAt: '2026-06-15T10:05:01.000Z', version: 1 };
}

describe('receiptVerification', () => {
  it('validates an intact, content-free receipt', async () => {
    expect(await validateReceipt(await receipt(), sha256Hex)).toEqual({ valid: true, reason: null, firstBrokenSeq: null
});
  });

  it('rejects a wrong version', async () => {
    const r = await receipt(); (r as any).version = 2;
    expect((await validateReceipt(r, sha256Hex)).reason).toBe('bad_version');
  });

  it('rejects forbidden content keys anywhere', () => {
    expect(findForbiddenKey({ summary: { note: { clipboard: 'x' } } })).toBe('summary.note.clipboard');
    expect(findForbiddenKey({ a: { token: 1 } })).toBe('a.token');
    expect(findForbiddenKey({ a: { ok: 1 } })).toBeNull();
  });

  it('rejects a receipt that smuggles a forbidden key', async () => {
    const r = await receipt(); (r as any).leaked = { keycode: 65 };
    expect((await validateReceipt(r, sha256Hex)).reason).toBe('forbidden_content');
  });

  it('rejects over-claiming summary counts', async () => {
    const r = await receipt();
    r.summary = { ...r.summary, counts: { ...r.summary.counts, inputBlocked: 999 } };
    expect(structuralCheck(r)).toBe('summary_mismatch');
  });

  it('rejects a tampered chain', async () => {
    const r = await receipt();
    r.lineItems[1] = { ...r.lineItems[1], type: 'emergency_stop.engaged' };
    const v = await validateReceipt(r, sha256Hex);
    expect(v.reason).toBe('chain_invalid');
    expect(v.firstBrokenSeq).toBe(1);
  });
});

```

FILE: tests/admin/receiptVerifier.service.test.ts

```

import { describe, it, expect, vi, beforeEach } from 'vitest';
import { createHash } from 'node:crypto';

const store = vi.fn();
const getJson = vi.fn();
vi.mock('../../apps/api/src/repositories/consentReceiptRepository', () => ({

```

```

    consentReceiptRepository: { store: (...a: unknown[]) => store(...a), getJson: (...a: unknown[]) => getJson(...a) },
  }));
vi.mock('../..../apps/api/src/lib/nodeSha256', () => ({
  nodeSha256Hex: async (i: string) => createHash('sha256').update(i).digest('hex'),
}));

import { receiptVerifierService } from '../..../apps/api/src/services/admin/receiptVerifier.service';
import { buildChain, makeAuditEvent, type ConsentReceipt, type SessionSummary } from '@remotedesk/shared';

const sha256Hex = async (i: string) => createHash('sha256').update(i).digest('hex');
const summary: SessionSummary = {
  deviceId: 'dev-1', startedAt: '2026-06-15T10:00:00.000Z', endedAt: '2026-06-15T10:01:00.000Z',
  durationSeconds: 60, capabilitiesUsed: { remoteInput: false, clipboardSync: false, fileTransfer: false },
  unattended: false, counts: { inputBlocked: 0, inputRateLimited: 0, emergencyStops: 0, controlGrants: 1, controlRevokes:
0 },
  endReason: 'user_disconnect',
};

async function goodReceipt(): Promise<ConsentReceipt> {
  const { lineItems, rootHash } = await buildChain([makeAuditEvent('control.granted')], sha256Hex);
  return { receiptId: 'r1', summary, lineItems, rootHash, issuedAt: '2026-06-15T10:01:01.000Z', version: 1 };
}

describe('receiptVerifierService', () => {
  beforeEach(() => { store.mockReset(); getJson.mockReset(); });

  it('stores a valid receipt', async () => {
    const r = await goodReceipt();
    const result = await receiptVerifierService.storeVerified('org', r);
    expect(result.valid).toBe(true);
    expect(store).toHaveBeenCalledWith('org', 'dev-1', r);
  });

  it('rejects + does not store a tampered receipt', async () => {
    const r = await goodReceipt();
    r.rootHash = 'deadbeef';
    await expect(receiptVerifierService.storeVerified('org', r)).rejects.toThrow();
    expect(store).not.toHaveBeenCalled();
  });

  it('verifyStored re-checks a stored receipt', async () => {
    getJson.mockResolvedValue(await goodReceipt());
    expect((await receiptVerifierService.verifyStored('org', 'r1')).valid).toBe(true);
  });

  it('verifyStored throws when not found', async () => {
    getJson.mockResolvedValue(null);
    await expect(receiptVerifierService.verifyStored('org', 'missing')).rejects.toThrow();
  });
});

```

Part G — Reports

FILE: [generated-pack24-web-api-rbac-receipt-verify-merge-summary.md](#)

Pack 24 (Web + API) — Admin RBAC + Server-Side Receipt Verification · Merge Summary

What this slice does

Closes the two gaps Pack 23 flagged:

1. Admin RBAC: only owner/admin may set device trust or create/revoke unattended grants. View stays open to any org member. Fail-closed on unknown role.
2. Server-side receipt verification: the API recomputes the consent receipt's hash chain (same algorithm as the desktop) AND runs content-free + anti-overclaim structural checks BEFORE storing. Tampered/over-claiming receipts are rejected. A new endpoint re-verifies stored receipts (tamper-check at rest).

Files created (9)

- packages/shared/src/permissions/adminRbac.ts
- packages/shared/src/permissions/receiptVerification.ts
- apps/api/src/middleware/requireDeviceAdmin.middleware.ts
- apps/api/src/services/admin/receiptVerifier.service.ts
- apps/api/src/lib/nodeSha256.ts
- apps/web/app/dashboard/devices/useDeviceAdminPermissions.ts
- tests/admin/adminRbac.test.ts
- tests/admin/receiptVerification.test.ts
- tests/admin/receiptVerifier.service.test.ts

Files modified (4)

- packages/shared/src/permissions/index.ts — export adminRbac + receiptVerification.
- apps/api/src/routes/deviceAdmin.routes.ts — requireDeviceAdmin on mutations; verify-on-store; add /receipts/:id/verify.
- apps/web/app/dashboard/devices/[deviceId]/page.tsx — hide trust/grant controls for non-admins.
- apps/web/app/dashboard/receipts/page.tsx — per-row verify badge.
- (also REVIEW_REQUIRED: errorCodes.ts — add RECEIPT_INVALID + RECEIPT_NOT_FOUND)

Files intentionally NOT touched

- apps/desktop/** — builds receipts identically; no change needed.
- apps/api/prisma/** — verification is computed, no schema change.

Remaining manual wiring risks

1. orgContext must populate orgRole (Pack 6) for requireDeviceAdmin to read.
2. Web page needs the caller's org role from your session/auth context.
3. Add the two error codes.
4. Confirm the desktop's SHA-256 preimage matches buildChain/linePreimage exactly (it does in Pack 22) so server re-verify agrees.

Verify

- pnpm --filter @remotedesk/shared build && pnpm -r test
- Manual: as a member, trust/grant buttons hidden + API returns 403; as admin they work. POST a receipt with an edited line -> 400 rejected; GET /receipts/:id/verify on a stored one -> intact.

FILE: [generated-pack24-web-api-rbac-receipt-verify-code-review.md](#)

Pack 24 (Web + API) — Code Review Notes

Strengths

- RBAC is pure + fail-closed and reuses the org role model; UI gate mirrors the server guard (server authoritative).
- Receipt verification reuses the Pack 22 chain verifier, so desktop-built and server-checked "intact" are identical.
- Content-free invariant enforced by a deep forbidden-key scan + anti-overclaim count check, BEFORE hashing.
- Tampered receipts are never stored; stored receipts can be re-verified at rest.
- Pure helpers + service (with repo mocks) fully unit-tested.

```
## Reviewer checklist
- [ ] orgContext sets orgRole; requireDeviceAdmin reads it.
- [ ] Web role source wired into useDeviceAdminPermissions.
- [ ] /receipts store path calls storeVerified (not the raw repo).
- [ ] errorCodes RECEIPT_INVALID + RECEIPT_NOT_FOUND added.
- [ ] Forbidden-key list matches your receipt schema (no false positives on legit fields).
```

```
## Safety
- Only admins/owners can change trust or mint/revoke grants.
- Receipts are provably intact + content-free before storage; tamper-checkable at rest.
- No schema change; host still owns live session permissions.
```

FILE: generated-pack24-web-api-rbac-receipt-verify-manifest.json

```
{
  "pack": "pack24-web-api-rbac-receipt-verify",
  "title": "Admin RBAC on Trust/Grant Mutations + Server-Side Receipt Verification",
  "actualFileCount": 13,
  "filesCreated": [
    "packages/shared/src/permissions/adminRbac.ts",
    "packages/shared/src/permissions/receiptVerification.ts",
    "apps/api/src/middleware/requireDeviceAdmin.middleware.ts",
    "apps/api/src/services/admin/receiptVerifier.service.ts",
    "apps/api/src/lib/nodeSha256.ts",
    "apps/web/app/dashboard/devices/useDeviceAdminPermissions.ts",
    "tests/admin/adminRbac.test.ts",
    "tests/admin/receiptVerification.test.ts",
    "tests/admin/receiptVerifier.service.test.ts"
  ],
  "filesModified": [
    "packages/shared/src/permissions/index.ts",
    "apps/api/src/routes/deviceAdmin.routes.ts",
    "apps/web/app/dashboard/devices/[deviceId]/page.tsx",
    "apps/web/app/dashboard/receipts/page.tsx"
  ],
  "filesNotTouched": [
    "apps/desktop/**",
    "apps/api/prisma/**",
    "apps/web/app/dashboard/devices/page.tsx"
  ],
  "dependenciesRequired": {
    "new": [],
    "existing": ["@remotedesk/shared", "express", "next", "react", "vitest"],
    "assumedFromProject": [
      "Pack 6 org role model + orgContext populating orgRole",
      "Pack 23 deviceAdmin routes + consentReceiptRepository",
      "Pack 22 consent receipt buildChain/verifyReceipt",
      "apps/web session/auth context exposing the caller's org role"
    ]
  },
  "knownLimitations": [
    "RBAC is owner/admin vs member; finer-grained device-admin roles are out of scope.",
    "Verification proves internal consistency + content-free shape, not cross-device signing (a later attestation step).",
    "Forbidden-key scan is heuristic; tune the list to your receipt schema."
  ]
}
```

```
    "Web role gating is UX only; the server guard is authoritative."
  ],
  "remainingManualWiringRisks": [
    "Ensure orgContext sets orgRole for requireDeviceAdmin.",
    "Wire the web caller's role into useDeviceAdminPermissions.",
    "Route /receipts store through storeVerified; add the two error codes."
  ],
  "estimatedCompletionAfterCarefulMerge": "78-84%"
}
```

Pack 24 (Web + API) done: 13 files (9 created, 4 modified). Closes both gaps Pack 23 flagged. Trust changes and grant create/revoke are now **owner/admin only** (members can still view), guarded server-side with a fail-closed RBAC middleware and mirrored in the web UI. And consent receipts are **verified server-side before storage**: the API recomputes the hash chain with the same algorithm the desktop used and runs content-free + anti-overclaim structural checks, so a tampered or over-claiming receipt is rejected with a 400 and never persisted. A new `/receipts/:id/verify` endpoint re-checks stored receipts at rest.

The clean bit: verification reuses Pack 22's exact `verifyReceipt` / `linePreimage`, so "intact" means the same thing on the desktop that built it and the server that checks it, no second implementation to drift. The content-free invariant is enforced by a deep forbidden-key scan that runs *before* hashing, so a receipt can't smuggle payloads/keystrokes/IPs past the API even if the chain is valid. No schema change, and the host still owns live session permissions.

That hardens the whole trust/consent system end to end. Next natural slice = device-admin audit (log who trusted/blocked/granted, via Pack 14), or cross-device receipt signing/attestation so receipts are provable beyond internal consistency. Bolo.